

ChatWithAI — AI-Assisted Code Optimization Record

Course: CSC3060 Project 5: Code Optimization

Date: 2026-05-16

Group Members: YIQI LI (124090324), JUNHONG JIANG (124090252)

AI Tool Used: Claude Code (Anthropic), deepseek-v4-pro model

Overview

This document records key interactions with the AI assistant during the optimization of 10 kernel functions. The AI was used for: identifying bottlenecks, proposing optimization strategies, debugging numerical correctness issues, and generating AVX2/AVX-512 optimized implementations. All final code was reviewed and validated by the group members.

1. GRFF — Gated Residual Feature Fusion

Initial Request

Asked AI to optimize `stu_grff` with the constraint that only the student function may be modified, output must match naive exactly.

Key Interactions

Attempt 1 — Aggressive fusion: AI proposed precomputing $B \cdot (1 - G)$ instead of $1 - G$, factoring $\text{half} = 0.5 \cdot \text{prod} \cdot \text{inv}$, fusing smooth+output into one pass with reverse in-place traversal, and algebraic simplification to $(C + \text{sig}) \cdot (1 - \text{sig}) - B_{\text{omg}} \cdot \text{avg_a} \cdot \text{inv_s}$.

Result: Output mismatched naive. The algebraic simplification changed floating-point rounding order.

Attempt 2 — Removed pragma, kept fusion: AI suspected `#pragma clang loop vectorize(enable)` caused fast reciprocal without Newton-Raphson refinement. Rewrote Pass 2 as fused 4x unrolled scalar code without pragmas.

Result: Still mismatched. The 4-way parallel sum $(\text{sum0} + \text{sum1} + \text{sum2} + \text{sum3}) / n$ produced a different `avg_a` than the naive's serial sum.

Attempt 3 — Exact formula matching: AI proposed matching every formula character-by-character with the naive. Changed `G` computation to $0.5f \cdot (p / (1 + |p|) + 1)$ (identical to naive line 36), used serial scalar sum, and kept Stages 5-9 in the naive's exact multi-step form (`b_prime` → `c_prime` → `h` → `e` → `result`) rather than algebraically simplified.

Result: Output correct. Verified by group.

Attempt 4 — AVX2 optimization: After correctness was confirmed, AI added AVX2 8-wide SIMD for `G` computation using `_mm256_div_ps` (precise division) and `_mm256_andnot_ps` (bitwise abs). Sum kept as separate scalar serial loop.

Key insight from AI: "Algebraic equivalence does not guarantee floating-point equivalence. The formula structure AND operation order must match the naive exactly."

2. Image Processing

Initial Request

Asked AI to optimize `stu_image_proc` while keeping output correct.

Key Interactions

Attempt 1 — Taylor series + branchless: AI proposed replacing `std::sinf/std::cosf` with Taylor polynomials ($\sin(x) \approx x \cdot (1 - x^2/6)$, $\cos(x) \approx 1 - x^2/2$), branchless clamp via `std::fmin/std::fmax`, and extending the importance_weight LUT to 6 entries to eliminate branches.

Result: Output mismatched. AI identified a **semantic bug**: the naive passes `compress_val` (HDR compressed value) to `complex_mask_logic`, but the student code used the raw luminance `gray` for both the mask branch comparison and the `sin` computation.

Fix: Changed `if (gray > threshold) → if (hdr > threshold)` and `sin(gray * p_sin) → sin(hdr * p_sin)`.

Key insight from AI: "The function parameter name `gray` inside `complex_mask_logic` is misleading — it receives `compress_val`, not the raw luminance. Trace the call site, not the parameter name."

3. Matrix Multiplication

Initial Request

Asked AI to optimize `stu_matmul` for $>2.45\times$ speedup over baseline, output must be correct.

Key Interactions

Attempt 1 — 3D tiling: AI proposed proper 3D tiling (`kk → ii → jj → i → k → j`) with BK=64, arguing the original code only tiled `(i, j)` without tiling `k`, providing zero cache benefit.

Result: Correct but not fast enough.

Attempt 2 — k-unrolling: AI added 2× k-unrolling (`c_row[j] += aik0*b0[j] + aik1*b1[j]`) and 4× j-unrolling to increase ILP.

Result: Output mismatched. AI identified that `C += (a+b) ≠ C += a; C += b` in floating-point — the pair-wise addition changed the accumulation order compared to the naive's sequential `sum += A[i][k]*B[k][j]`.

Fix: Removed k-unrolling entirely. Each product is added one at a time, matching the naive's sequential accumulation.

Attempt 3 — Multi-threading: AI added a `std::thread` pool splitting rows across up to 16 workers. Each thread writes to disjoint output rows (no synchronization needed).

Key insight from AI: "k-unrolling changes the addition order because it combines two products before adding to the accumulator. The naive adds one product at a time to a local `sum` variable. These are not numerically equivalent."

4. Filter Gradient

Initial Request

Asked AI to analyze and optimize `stu_filter_gradient`.

Key Interactions

AI proposed: 4× unrolling with overlap elimination. Adjacent pixels' 3×3 windows share 2 columns, so 18 struct reads serve 4 pixels instead of 36. Also added 4 independent `double` accumulators (`total0..total3`) to break the dependency chain.

Result: Correct, ~2× memory traffic reduction.

Key insight from AI: "A 4× unrolled sliding window needs 6 columns × 3 rows = 18 Pixel structs pre-loaded. These 18 structs serve all 4 output pixels through index shifting (pixel 0 uses columns 0-2, pixel 1 uses columns 1-3, etc.)."

5. Black-Scholes

Initial Request

Asked AI to optimize `stu_BlackSchls`.

Key Interactions

AI proposed: Removing `#pragma omp simd` (silently ignored without `-fopenmp` in the build), replacing with 4× manual unrolling. Extracted `bs_cndf` as a `static inline` helper. Horner's method already optimal for the polynomial.

Result: Correct, better ILP. 4 independent options per iteration let the CPU interleave `expf/sqrtf` calls, hiding their 10-20 cycle latency.

Key insight from AI: "The `#pragma omp simd` directive is silently ignored when the compiler is invoked without `-fopenmp`. Manual unrolling is the portable alternative."

6. Sparse SpMM

Initial Request

Asked AI to further optimize `stu_csr_spmm`.

Key Interactions

AI proposed two changes:

1. **Eliminated per-row `memset`**: The first nonzero in each row uses `=` (assignment) to cover all output columns. Subsequent nonzeros use `+=`. Empty rows still get a single `memset`. Saves ~16MB of redundant writes.
2. **Increased unrolling to 16x**: Aligns with AVX-512 width for `vfmadd231ps` instructions.

Result: Correct. Verified by group.

7. Bitwise

Initial Request

Asked AI to optimize `stu_bitwise`.

Key Interactions

AI proposed: Replacing the lambda closure with a `static inline process_chunk()` function, increasing unrolling from 4x to 8x (64 bytes/iteration), and using `__builtin_memcpy` instead of `std::memcpy` (guaranteed inline at `-O2`).

Key insight from AI: "`std::memcpy` with a small constant size may or may not be inlined at `-O2`. `__builtin_memcpy` is a compiler intrinsic that is always inlined, avoiding function call overhead in the hot loop."

8. Graph

Initial Request

AI was asked to review and suggest optimizations.

Key Interactions

AI identified: The naive's pointer-chasing traversal (`e = e->next`) causes unpredictable memory access. The CSR conversion (already done outside timing) enables contiguous `to[]` array access with hardware prefetching. The 4-way accumulator unrolling on the checksum breaks the integer addition dependency chain.

9–10. ReLU & Trace Replay

ReLU

AI confirmed the 16x unrolling was appropriate for AVX-512 auto-vectorization to `vmaxps`. Element-wise operations are memory-bandwidth-bound, so wider unrolling saturates cache bandwidth.

Trace Replay

AI confirmed the precomputed cost table approach was optimal — trading 512KB of storage to eliminate ~5M redundant arithmetic operations across 1M trace lookups to 65K records.

Summary of Key Lessons from AI Interactions

1. **Floating-point algebra $\neq$ floating-point arithmetic.** Algebraic simplifications that are mathematically correct can produce different results due to rounding order. The exact formula structure (including addition order and parentheses) matters when tolerances are tight (1e-6 to 1e-5).
2. **Vector accumulators change sum order.** Parallel reduction via `_mm256_add_ps(vsum, ...)` produces a different result than scalar serial accumulation. Use scalar accumulation for sums that feed into subsequent computations with tight error tolerances.
3. **SIMD fast-reciprocal instructions (~12-bit) need Newton-Raphson refinement** when full 23-bit precision is required. Use `_mm256_div_ps` (precise) or `_mm256_rcp_ps` + NR step explicitly.
4. **Trace data flow through function calls.** Misleading parameter names (e.g., `gray` receiving `compress_val`) can cause semantic bugs. Always verify the call site.
5. **Compiler pragmas need matching flags.** `#pragma omp simd` needs `-fopenmp`. `#pragma clang loop vectorize(enable)` may be ignored if the loop has dependencies that prevent vectorization.
6. **Memory traffic reduction is often the biggest win.** Techniques like overlap-aware unrolling (Filter Gradient), scratch buffer reuse (GRFF), and eliminating redundant zeroing (Sparse SpMM) reduce memory bandwidth consumption, the primary bottleneck on modern CPUs.